| | ITER, SIKSHA 'O' ANUSANDHAN (Deemed to be University) | | Assignment 1 |
|---|---|---|---|
| Branch | CSIT | Programme | **B.Tech** |
| Course Name | Full-Stack Web Development with MERN | Semester | 6th |
| Course Code | CSE 3839 | Academic Year | 2025-2026 |
| **Lab Assignment: 2** | | **Topic: Programs on HTTP server Nodejs** | |

| Learning Level (LL) | **L1**: Remembering | **L3:** Applying | **L5**: Evaluating |
|---|---|---|---|
| | **L2**: Understanding | **L4**: Analysing | **L6**: Creating |

| Q's | Questions | COs | LL |
|---|---|---|---|
| **1.** | Create a file data.csv and store student name and marks in CSV format.This program uses the 'fs' module to create and write data into a CSV file. CSV (Comma Separated Values) format stores data separated by commas. Use writeFileSync() to create the file and insert formatted text. | **CO2** | **L2 L3** |
| **2.** | Read data from data.csv and display it.This program reads file content using readFileSync(). The 'utf8' encoding converts buffer data into readable text format. | **CO2** | **L3 L4** |
| **3.** | Write a Node.js program to create a basic HTTP server that displays an HTML heading in the browser.<br><br>Your program should:<br>• Import the required http module.<br>• Create a server using the http.createServer() method.<br>• Set the HTTP response status code to 200 (OK).<br>• Define the response header with content type as text/html.<br>• Send an HTML response containing a heading (for example, <h1>Hello Students</h1>).<br>• Properly end the response using res.end().<br>• Run the server on port 4000.<br>• Display a confirmation message in the terminal indicating that the server is running.<br>• Also mention the steps required to execute the program and view the output in the browser. | **CO2** | **L3,L4** |
| **4.** | Write a Node.js program that performs logging functionality by storing the current system date and time into a file named log.txt every time the program | **CO2** | **L3, L4** |

| | | | |
|---|---|---|---|
| | is executed.<br><br>Your program should:<br>• Import the required File System (fs) module.<br>• Generate the current date and time using the built-in Date object.<br>• Append the generated timestamp into a file named log.txt.<br>• Ensure that previous log entries are not deleted (i.e., data should be added, not overwritten).<br>• Add each new timestamp on a new line for proper readability.<br>• Display a confirmation message in the terminal after updating the log file. | | |
| **5** | Create a Node.js HTTP server that performs the following tasks:<br>• Import the required http module.<br>• Create a server using http.createServer().<br>• Set the response header with status code 200 and content type as application/json.<br>• Create a JavaScript object containing student details (name and marks).<br>• Convert the JavaScript object into JSON format using JSON.stringify().<br>• Send the JSON data as a response to the browser.<br>• Run the server on port 5000.<br>• Display a message in the terminal confirming that the server is running. | **CO2** | **L3,L4** |
| **6** | Create a Node.js HTTP server that retrieves student details from MongoDB and sends the data in JSON format to the browser. Create a database School and collection as Student. Initialize Node.js with npm initi-y and install mongodb. Create a file student.js for creating HTTP server using Node.js that connects to MongoDB (NoSQL database) retrieves student data using a query, converts it into JSON format, and sends it as a response. Create a docker container dbserver and pull mongodb. Perform following operations on the database.<br>• Insert one student document with fields :Name,age,class,marks.<br>• Insert multiple student documents.<br>• Display all student records.<br>• Find a student by name.<br>• Update the marks of a particular student. | **CO2** | **L3,L4** |

| | | | |
|---|---|---|---|
| | • Delete a student record.<br>• Display the list of databases.<br>• Display the list of collections in the current database. | | |
| **7** | Create a NoSQL database named ecommerce in MongoDB and create a collection called product.<br>• Insert one product document with the fields name, price, category, and stock.<br>• Insert multiple product documents into the collection.<br>• Retrieve and display all product records stored in the database.<br>• Find a specific product by its name.<br>• Update the price or stock of a particular product. Delete a product record from the collection.<br>• Display the list of all databases and show the collections present in the current ecommerce database.<br>Further, create a Node.js HTTP server using the built-in http module that connects to MongoDB using the mongodb package. The server should retrieve product data from the product collection using an appropriate MongoDB query, convert the retrieved documents into JSON format using JSON.stringify(), set the response header as application/json, and send the data as a response to the browser. The server should run on a specified port and display a confirmation message in the terminal indicating that the server is running successfully.<br><br>Write the required MongoDB queries and Node.js code necessary to perform the above operations. | | |

1. Assignment carries a weightage of 20 marks out of 100
2. Only CO2 outcomes are covered.

| | | |
|---|---|---|
| Course Outcomes | CO1 | Understand and configure a modern full-stack development environment using Node.js, MongoDB, Docker, and supporting tools. |
| | CO2 | Design and implement RESTful backend services using Express, MongoDB (Mongoose), and unit testing frameworks. |
| | CO3 | Develop interactive frontend applications using React and efficiently integrate them with backend APIs using modern data-fetching techniques. |
| | CO4 | Apply DevOps practices to containerize, test, and deploy full-stack applications using Docker and CI/CD pipelines. |

| | | |
|---|---|---|
| | CO5 | Implement authentication, performance optimization, Search Engine Optimization techniques, and automated testing for scalable and secure web applications. |
| | CO6 | Build and integrate advanced full-stack architectures using data aggregation, visualization, and GraphQL APIs with modern frontend clients. |